

A Practical Guide to Backpropagation

TOPIC -- BACKPROPAGATION

$$\frac{\partial L}{\partial W} = \delta \cdot a^T$$

$$\delta^{(l)} = (W^{(l+1)T} \delta^{(l+1)}) \odot \sigma'(z^{(l)})$$

-- 01 --

Derivation The gradient descent method involves calculating the derivative of the loss function with respect to the weights of the network. This is normally done using backpropagation. Assuming one output neuron, the squared error function is

-- 02 --

Then, the loss function E takes the form of a parabolic cylinder with its base directed along $w_1 = -w_2$. Since all sets of weights that satisfy $w_1 = -w_2$ minimize the loss function, in this case additional constraints are required to converge to a unique solution. Additional constraints could either be generated by setting specific conditions to the weights, or by injecting additional training data. One commonly used algorithm to find the set of weights that minimizes the error is gradient descent. By backpropagation, the steepest descent direction is calculated of the loss function versus the present synaptic weights. Then, the weights can be modified along the steepest descent direction, and the error is minimized in an efficient way.

-- 03 --

The gradients of the weights can thus be computed using a few matrix multiplications for each level; this is backpropagation. Compared with naively computing forwards (using the δ^l for illustration):

-- 04 --

a neural network trained to drive autonomously using backpropagation. The LeNet was published in 1989 to recognize handwritten zip codes. In 1992, TD-Gammon achieved top human level play in backgammon. It was a reinforcement learning agent with a neural network with two layers, trained by backpropagation. In 1993, Eric Wan won an international pattern recognition contest through backpropagation.

-- 10 --

Note that δ^l is a vector, of length equal to the number of nodes in level l ; each component is interpreted as the "cost attributable to (the value of) that node". The gradient of the weights in layer l is then:

-- 11 --

Using a Hessian matrix of second-order derivatives of the error function, the Levenberg–Marquardt algorithm often converges faster than first-order gradient descent, especially when the topology of the error function is complicated. It may also find solutions in smaller node counts for which other methods might not converge. The Hessian can be approximated by the Fisher information matrix. As an example, consider a simple feedforward network. At the l -th layer, we have x_i^l , $a_i^l = f(x_i^l)$, $x_i^{l+1} = \sum_j W_{ij}^l a_j^l$ where x are the pre-activations, a are the activations, and W is the weight matrix. Given a loss function L , the first-order backpropagation states that $\frac{\partial L}{\partial a_j^l} = \sum_i W_{ij}^l \frac{\partial L}{\partial x_i^{l+1}}$, $\frac{\partial L}{\partial x_j^l} = f'(x_j^l) \frac{\partial L}{\partial a_j^l}$ and the second-order backpropagation states that $\frac{\partial^2 L}{\partial a_{j_1}^l \partial a_{j_2}^l} = \sum_{i_1, i_2} W_{i_1 j_1}^l W_{i_2 j_2}^l \frac{\partial^2 L}{\partial x_{i_1}^{l+1} \partial x_{i_2}^{l+1}}$, $\frac{\partial^2 L}{\partial x_{j_1}^l \partial x_{j_2}^l} = f'(x_{j_1}^l) f'(x_{j_2}^l) \frac{\partial^2 L}{\partial a_{j_1}^l \partial a_{j_2}^l} + \delta_{j_1 j_2} f''(x_{j_1}^l) \frac{\partial L}{\partial a_{j_1}^l}$ where δ is the Dirac delta symbol. Arbitrary-order derivatives in arbitrary computational graphs can

be computed with backpropagation, but with more complex expressions for higher orders.